



WEATHER MANAGEMENT SYSTEM

OOP LAB PROJECT

GROUP MEMBERS:

Khadija Imran (2025-AI-193)

Esha Noor Qamar (2025-AI-196)

Iman Fatima (2025-AI-183)

Alishba (2025-AI-188)

DEPARTMENT:

Electrical Engineering Department

SUBMITTED TO:

Sir Salman

SUBMISSION DATE:

6th- June - 2026

Abstract:

Weather forecasting and weather data management are important for daily life, agriculture, transportation, and disaster management. The objective of this project is to develop a Weather Management System using C++ programming language that allows users to manage and analyze weather-related information efficiently.

The system is a console-based application developed using C++. It provides facilities for storing weather records, displaying weather information, updating data, searching records, and generating weather reports. The project utilizes fundamental programming concepts such as functions, arrays, file handling, conditional statements, loops, and user-defined structures. The developed system provides a simple and efficient solution for managing weather data while improving data accessibility and organization.

Table of Contents:

01	Introduction
02	Background
03	Related Work
04	Project Management
05	System Design and Methodology
06	Implementation Details
07	Project Results and Analysis
08	Challenges Faced
09	Final Budget
10	Project Deliverables
11	Recommendations and Future Work
12	Conclusion
13	References
14	Appendix

1. Introduction:

Problem Statement:

Managing weather information manually can be difficult and time-consuming. There is a need for a computerized system that can store, update, search, and display weather data efficiently.

Motivation:

Weather information is required in many fields such as agriculture, transportation, tourism, and disaster management. A digital system helps reduce errors and improve accessibility.

Challenges:

- Managing multiple weather records.
- Ensuring accurate data storage.
- Efficient searching and updating of information.
- User-friendly interaction through a console interface.

Proposed Solution:

A console-based Weather Management System was developed using C++ that enables users to add, update, delete, search, and display weather records.

Assumptions:

- Users enter correct weather information.
- Data is stored locally.
- The application runs on a standard C++ compiler.

Main Contributions:

- Developed a complete weather management application.
- Implemented data management functionalities.
- Provided efficient record searching and updating.
- Applied C++ programming concepts in a real-world application.

2. Background:

Weather management systems are software applications used to collect, store, and process weather-related information. Such systems assist users in maintaining organized records and generating useful reports.

The project uses:

- C++ Programming Language
- Functions

- Arrays
- Structures
- File Handling
- Loops and Conditional Statements

These concepts help in creating a structured and efficient application.

3. Related Work:

Various weather applications and forecasting systems exist today. Most modern systems use cloud computing, sensors, and artificial intelligence.

Compared to advanced systems, this project:

- Is lightweight and easy to use.
- Operates through a console interface.
- Requires minimal hardware resources.
- Focuses on weather record management rather than advanced forecasting.

4. Project Management:

Task Breakdown:

Task | Description

Requirement Analysis | Understanding project requirements

System Design | Designing program structure

Coding | Implementing functionalities

Testing | Checking for errors

Documentation | Preparing final report

Project Timeline:

Week 1: Requirement Analysis

Week 2: System Design

Week 3: Coding

Week 4: Testing

Week 5: Documentation

5. System Design and Methodology:

System Features:

- Add Weather Record
- View Weather Records
- Search Weather Data
- Update Weather Information
- Delete Weather Records
- Generate Reports

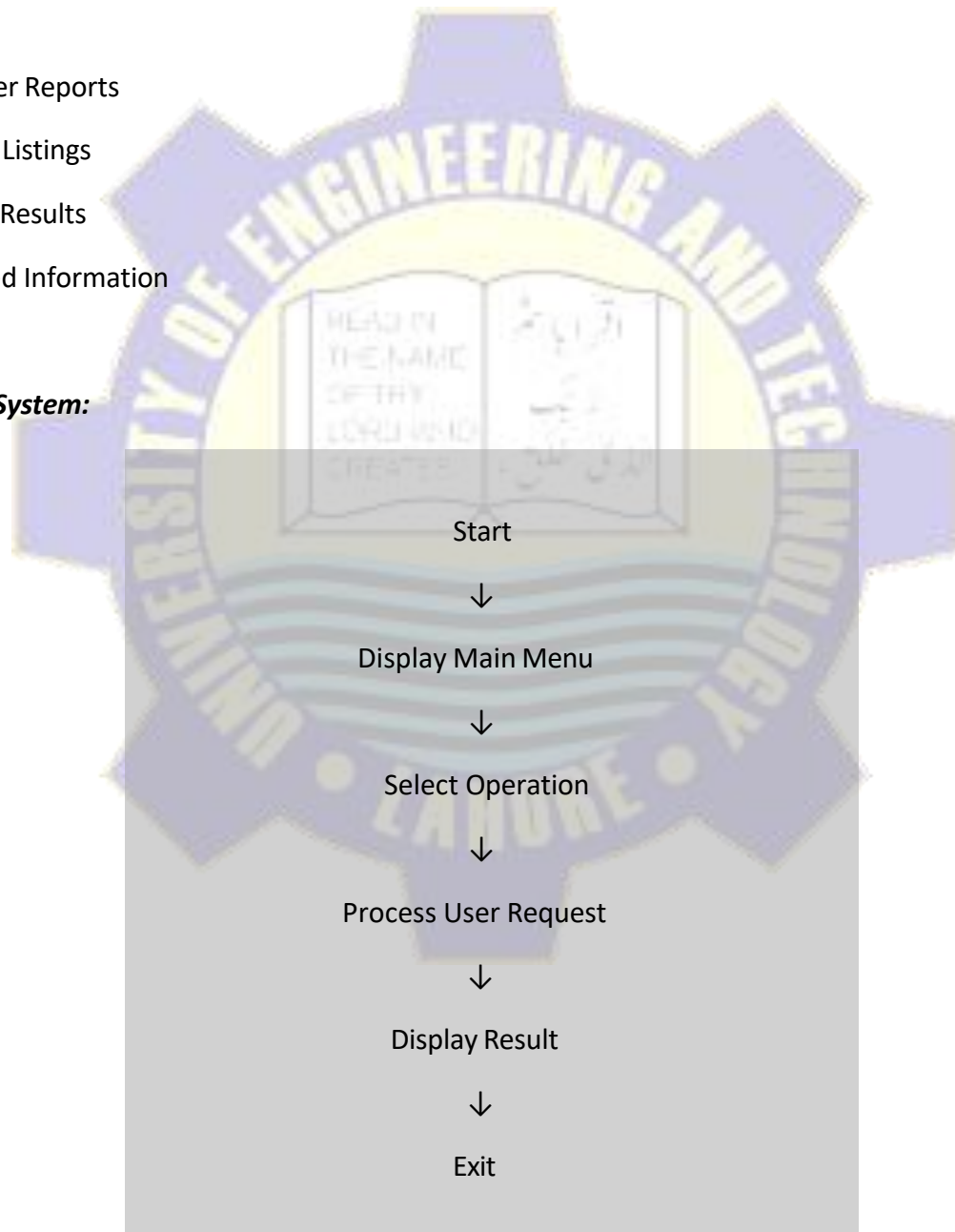
Input Data:

- Date
- Temperature

- Humidity
- Wind Speed
- Weather Condition

Output Data:

- Weather Reports
- Record Listings
- Search Results
- Updated Information

Flow of System:

6. Implementation Details:

The project was implemented using C++.

- Main Modules
- Add Record Module
- Allows users to enter weather information and save records.
- Search Module
- Searches weather records using date or weather condition.
- Update Module
- Modifies existing weather information.
- Delete Module
- Removes unwanted records.
- Display Module
- Shows all stored weather records.

Programming Concepts Used:

- Functions
- Structures
- Arrays
- File Handling
- Loops
- Conditional Statements

7. Project Results and Analysis:

The Weather Management System successfully performs all required operations.

Results:

- Accurate storage of weather records.
- Fast retrieval of data.
- Easy modification and deletion of records.
- User-friendly menu-driven interface.

Performance Analysis:

Feature | Status

Add Record | Successful

Search Record | Successful

Update Record | Successful

Delete Record | Successful

Display Records | Successful

The system achieved all planned objectives and operated correctly during testing.

8. Challenges Faced:**Challenge 1:**

Managing file handling operations.

Solution: Implemented proper file reading and writing techniques.

Challenge 2:

Handling invalid user input.

Solution: Added validation checks.

Challenge 3:

Debugging program logic.

Solution: Conducted repeated testing and error correction.

9. Final Budget:

Item | Cost

Software Tools | Free

C++ Compiler | Free

Documentation | Free

Total Cost | 0 PKR

The project was developed using freely available software tools.

10. Project Deliverables:

The following deliverables were completed:

- Weather Management System Source Code
- Console-Based Application
- Project Documentation

- Final Report
- Testing Results

11. Recommendations and Future Work:

Future improvements may include:

- Graphical User Interface (GUI)
- Real-Time Weather API Integration
- Database Connectivity
- Online Access
- Weather Forecasting Using Machine Learning
- Data Visualization Charts

12. Conclusion:

The Weather Management System was successfully designed and implemented using C++. The system provides efficient management of weather information through various functionalities such as adding, searching, updating, deleting, and displaying weather records. The project demonstrates practical implementation of programming concepts and provides a reliable solution for weather data management.

13. References:

- Bjarne Stroustrup, Programming: Principles and Practice Using C++.
- C++ Documentation.
- Weather Data Management System Research Articles.
- Online C++ Programming Tutorials.

14. Appendix:

Sample Weather Record:

Date: 05-06-2026

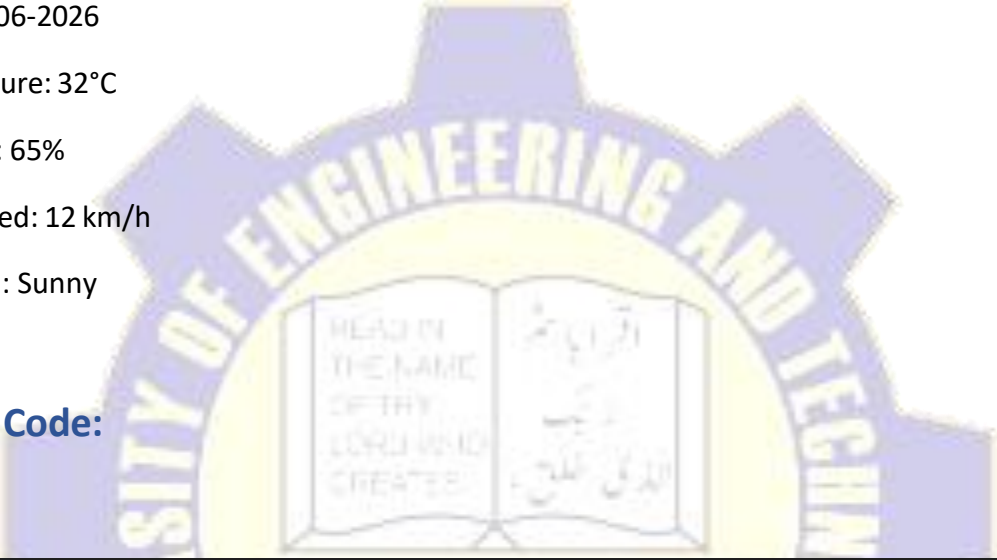
Temperature: 32°C

Humidity: 65%

Wind Speed: 12 km/h

Condition: Sunny

Source Code:



```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
#include <iomanip>
#include <algorithm>
#include <cstdlib>
#include <ctime>
#include <cstdio>

using namespace std;
```

```
// =====
// API CONFIGURATION
// =====

const string API_KEY = "183f18fd290f1bfa94ec326e5228af3b";
const string API_URL = "http://api.openweathermap.org/data/2.5/weather?q=";

// =====
// CLASS: Location
// =====

class Location {
private:
    int locationId;
    string cityName;
    string country;

public:
    Location() : locationId(0), cityName(""), country("") {}
    Location(int id, string city, string country)
        : locationId(id), cityName(city), country(country) {}

    int getLocationId() const { return locationId; }
    string getCityName() const { return cityName; }
    string getCountry() const { return country; }

    void setCityName(const string& city) { cityName = city; }
    void setCountry(const string& country) { this->country = country; }
```

```

    string getCoordinates() const {
        return "Lat: " + to_string(locationId * 10) + ", Long: " +
to_string(locationId * 20);
    }

    void displayLocation() const {
        cout << "Location ID: " << locationId << endl;
        cout << "City: " << cityName << ", Country: " << country << endl;
        cout << "Coordinates: " << getCoordinates() << endl;
    }
};

// =====
// CLASS: Alert
// =====

class Alert {
private:
    string alertType;
    int severity;

public:
    Alert() : alertType(""), severity(0) {}
    Alert(string type, int sev) : alertType(type), severity(sev) {}

    string getAlertType() const { return alertType; }
    int getSeverity() const { return severity; }

    void setAlertType(const string& type) { alertType = type; }

```



```

void setSeverity(int sev) { severity = sev; }

void sendNotification() const {
    cout << "\n===== " << endl;
    cout << "          WEATHER ALERT!" << endl;
    cout << "===== " << endl;
    cout << "Type: " << alertType << endl;
    cout << "Severity Level: " << severity << "/5" << endl;
    if (severity >= 4) {
        cout << "WARNING: Extreme weather conditions detected!" << endl;
    }
    cout << "===== " << endl;
}
};

// =====
// CLASS: WeatherReport (Abstract Base Class)
// =====

class WeatherReport {
protected:
    string date;
    float temperature;
    int humidity;
    float windSpeed;
    int pressure;
    bool isLiveData;

```

```

public:
    WeatherReport() : date(""), temperature(0.0), humidity(0), windSpeed(0.0),
pressure(0), isLiveData(false) {}

    WeatherReport(string d, float temp, int hum, float wind, int pres)
        : date(d), temperature(temp), humidity(hum), windSpeed(wind),
pressure(pres), isLiveData(false) {}

    virtual void display(bool isMetric) const = 0;

    string getDate() const { return date; }
    float getTemperature() const { return temperature; }
    int getHumidity() const { return humidity; }
    float getWindSpeed() const { return windSpeed; }
    int getPressure() const { return pressure; }
    bool getIsLiveData() const { return isLiveData; }

    void setDate(const string& d) { date = d; }
    void setTemperature(float temp) { temperature = temp; }
    void setHumidity(int hum) { humidity = hum; }
    void setWindSpeed(float wind) { windSpeed = wind; }
    void setPressure(int pres) { pressure = pres; }
    void setIsLiveData(bool live) { isLiveData = live; }

    virtual ~WeatherReport() {}
};

// =====
// CLASS: CurrentWeather (Derived Class)

```

```
// =====
class CurrentWeather : public WeatherReport {
private:
    string cityName;
    bool isExtreme;
    string condition;

public:
    CurrentWeather() : WeatherReport(), cityName(""), isExtreme(false),
condition("Clear") {}

    CurrentWeather(string city, string d, float temp, int hum, float wind, int
pres, string cond = "Clear")
        : WeatherReport(d, temp, hum, wind, pres), cityName(city),
isExtreme(false), condition(cond) {
        checkExtremeWeather();
    }

    void checkExtremeWeather() {
        if (temperature > 40.0 || temperature < 0.0 || windSpeed > 50.0) {
            isExtreme = true;
        } else {
            isExtreme = false;
        }
    }

    void display(bool isMetric) const override {
        cout << "\n===== " << endl;
        if (isLiveData) cout << "    LIVE DATA: " << cityName << " | " << date <<
endl;
```

```

else cout << " ?? CACHED DATA: " << cityName << " | " << date << endl;

cout << "===== " << endl;

float displayTemp = temperature;
string unit = "°C";
if (!isMetric) {
    displayTemp = (temperature * 9.0/5.0) + 32.0;
    unit = "°F";
}

cout << "Temperature: " << fixed << setprecision(1) << displayTemp << " "
<< unit << endl;

cout << "Condition: " << condition << endl;
cout << "Humidity: " << humidity << "%" << endl;
cout << "Wind Speed: " << windSpeed << " km/h" << endl;
cout << "Pressure: " << pressure << " hPa" << endl;

cout << "===== " << endl;

if (isExtreme) {
    Alert alert("Extreme Temperature", 5);
    alert.sendNotification();
}

}

string getCityName() const { return cityName; }
string getCondition() const { return condition; }
bool getIsExtreme() const { return isExtreme; }

```

```

void setCityName(const string& city) { cityName = city; }

void setCondition(const string& cond) { condition = cond; }

bool operator>(const CurrentWeather& other) const {
    return this->temperature > other.temperature;
}

bool operator<(const CurrentWeather& other) const {
    return this->temperature < other.temperature;
}

bool operator==(const CurrentWeather& other) const {
    return this->temperature == other.temperature;
}
};

// =====
// CLASS: ForecastWeather (Derived Class)
// =====

class ForecastWeather : public WeatherReport {
private:
    string cityName;
    string forecastDay;
    string condition;

public:
    ForecastWeather() : WeatherReport(), cityName(""), forecastDay(""),
condition("") {}

```

```

ForecastWeather(string city, string day, string d, float temp, int hum, float
wind, int pres, string cond)

    : WeatherReport(d, temp, hum, wind, pres), cityName(city),
forecastDay(day), condition(cond) {}

void display(bool isMetric) const override {

    cout << "\n-----" << endl;

    cout << "Forecast for " << cityName << " - " << forecastDay << endl;
    cout << "Date: " << date << endl;
    cout << "Condition: " << condition << endl;

    float displayTemp = temperature;
    string unit = "°C";
    if (!isMetric) {
        displayTemp = (temperature * 9.0/5.0) + 32.0;
        unit = "°F";
    }

    cout << "Temperature: " << fixed << setprecision(1) << displayTemp << " "
<< unit << endl;
    cout << "Humidity: " << humidity << "%" << endl;
    cout << "Wind: " << windSpeed << " km/h" << endl;
    cout << "Pressure: " << pressure << " hPa" << endl;
    cout << "-----" << endl;
}

string getCityName() const { return cityName; }
string getForecastDay() const { return forecastDay; }

```

```

    string getCondition() const { return condition; }

    void setCityName(const string& city) { cityName = city; }
    void setForecastDay(const string& day) { forecastDay = day; }
    void setCondition(const string& cond) { condition = cond; }
};

// =====
// CLASS: Favourite
// =====

class Favourite {
private:
    int favId;
    vector<string> favouriteCities;

public:
    Favourite() : favId(0) {}
    Favourite(int id) : favId(id) {}

    void addFavourite(const string& cityName) {
        for (const string& city : favouriteCities) {
            if (city == cityName) {
                cout << cityName << " is already in favourites!" << endl;
                return;
            }
        }
        favouriteCities.push_back(cityName);
    }
};

```



```

        cout << cityName << " added to favourites!" << endl;
    }

    void removeFavourite(const string& cityName) {
        for (auto it = favouriteCities.begin(); it != favouriteCities.end();
++it) {
            if (*it == cityName) {
                favouriteCities.erase(it);
                cout << cityName << " removed from favourites!" << endl;
                return;
            }
        }
        cout << cityName << " not found in favourites!" << endl;
    }

    bool isFavourite(const string& cityName) const {
        for (const string& city : favouriteCities) {
            if (city == cityName) return true;
        }
        return false;
    }

    void displayFavourites() const {
        cout << "\n--- Your Favourite Cities ---" << endl;
        if (favouriteCities.empty()) {
            cout << "No favourite cities added yet." << endl;
        } else {
            for (size_t i = 0; i < favouriteCities.size(); i++) {

```

```

        cout << (i + 1) << ". " << favouriteCities[i] << endl;

    }

}

cout << "-----" << endl;

}

vector<string> getFavourites() const { return favouriteCities; }

int getFavId() const { return favId; }

void setFavId(int id) { favId = id; }

};

// =====
// CLASS: User (Base Class)
// =====

class User {

protected:

    int userId;

    string username;

    string password;

    bool isLoggedIn;

    Favourite favourites;

public:

    User() : userId(0), username(""), password(""), isLoggedIn(false) {}

    User(int id, string user, string pass)
        : userId(id), username(user), isLoggedIn(false) {

```

```

        password = encryptPassword(pass);
    }

    string encryptPassword(const string& pass) const {
        string encrypted = pass;
        reverse(encrypted.begin(), encrypted.end());

        return encrypted;
    }

    void logout() {
        isLoggedIn = false;
        cout << "\nLogged out successfully." << endl;
    }

    bool getLoginStatus() const { return isLoggedIn; }
    void setLoginStatus(bool status) { isLoggedIn = status; }
    string getUsername() const { return username; }
    int getUserId() const { return userId; }
    Favourite& getFavourites() { return favourites; }

    void setUsername(const string& user) { username = user; }
    void setPassword(const string& pass) { password = encryptPassword(pass); }

    virtual void showMenu() const {
        cout << "\n===== USER MENU =====" << endl;

        cout << "1. View Current Weather (Real-Time)" << endl;

        cout << "2. View 5-Day Forecast" << endl;
    }

```

```

        cout << "3. Manage Favourites" << endl;

        cout << "4. Compare Cities" << endl;

        cout << "5. Generate Historical Report" << endl;

        cout << "6. Logout" << endl;

        cout << "======" << endl;
    }

    virtual ~User() {}
};

// =====
// CLASS: Admin (Inherits from User)
// =====

class Admin : public User {
private:
    int adminLevel;

public:
    Admin() : User(), adminLevel(1) {}

    Admin(int id, string user, string pass, int level = 1)
        : User(id, user, pass), adminLevel(level) {}

    void manageDataSources() {

        cout << "\n===== DATA SOURCE MANAGEMENT =====" << endl;

        cout << "1. Connect to OpenWeatherMap API" << endl;

        cout << "2. Sync Data Sources" << endl;

        cout << "3. Check API Status" << endl;
    }
};

```

```

        cout << "======" << endl;

        int choice;

        cout << "Enter choice: ";

        cin >> choice;

        switch(choice) {
            case 1: cout << "\n[Admin] Connecting to OpenWeatherMap
API...\n[Admin] Connection established!" << endl; break;

            case 2: cout << "\n[Admin] Synchronizing data sources...\n[Admin] All
data synchronized!" << endl; break;

            case 3: cout << "\n[Admin] API Status: ONLINE | Key: " <<
API_KEY.substr(0, 5) << "..." << endl; break;

            default: cout << "Invalid choice!" << endl;
        }
    }

    void generateHistoricalReport() {
        cout << "\n===== HISTORICAL WEATHER REPORT =====" << endl;
        cout << "Generating report for the last 7 days..." << endl;

        string days[] = {"Day 1", "Day 2", "Day 3", "Day 4", "Day 5", "Day 6",
"Day 7"};

        for (int i = 0; i < 7; i++) {
            cout << days[i] << ": Avg Temp 25°C, Humidity 60%" << endl;
        }

        cout << "======" << endl;
    }

    void showMenu() const override {

```

```

        cout << "\n===== ADMIN MENU =====" << endl;

        cout << "1. View All Weather Data" << endl;

        cout << "2. Manage Data Sources" << endl;

        cout << "3. Generate Historical Report" << endl;

        cout << "4. Manage Users" << endl;

        cout << "5. System Settings" << endl;

        cout << "6. Logout" << endl;

        cout << "===== " << endl;

    }

    int getAdminLevel() const { return adminLevel; }
};

// =====
// CLASS: WeatherAPI (Real API Integration)
// =====

class WeatherAPI {
private:
    bool isConnected;

public:
    WeatherAPI() : isConnected(false) {}

    bool connect() {
        cout << "\n[API] Connecting to OpenWeatherMap..." << endl;

        isConnected = true;

        cout << "[API] Connected successfully!" << endl;
    }
};

```

```

        return true;
    }

    string extractJsonValue(const string& json, const string& key) {
        string searchKey = "\"" + key + "\":\"";
        size_t pos = json.find(searchKey);

        if (pos == string::npos) return "";

        pos += searchKey.length();
        while (pos < json.length() && (json[pos] == ' ' || json[pos] == '\n'))
            pos++;

        if (json[pos] == '"') {
            pos++;
            size_t endPos = json.find("\"", pos);
            return json.substr(pos, endPos - pos);
        } else {
            size_t endPos = pos;
            while (endPos < json.length() && json[endPos] != ',' && json[endPos]
                != '}' && json[endPos] != ']') endPos++;
            return json.substr(pos, endPos - pos);
        }
    }

    bool fetchData(const string& cityName, float& temp, int& humidity,
        float& windSpeed, int& pressure, string& condition) {

```

```

    cout << "\n[API] Fetching REAL-TIME data for " << cityName << "..." <<
endl;

    string url = API_URL + cityName + "&appid=" + API_KEY + "&units=metric";
    string cmd = "curl -s \"" + url + "\" -o temp_api.json";
    int result = system(cmd.c_str());

    if (result != 0) {
        cout << "[API] Curl command failed. Falling back to cache." << endl;
        return false;
    }

    ifstream file("temp_api.json");
    if (!file.is_open()) {
        cout << "[API] Could not read response file." << endl;
        return false;
    }

    string jsonStr((istreambuf_iterator<char>(file)),
istreambuf_iterator<char>());
    file.close();

    if (jsonStr.find("\"cod\":404") != string::npos) {
        cout << "[API] City not found. Check spelling." << endl;
        return false;
    }

    string tempStr = extractJsonValue(jsonStr, "temp");
    string humStr = extractJsonValue(jsonStr, "humidity");

```



```

    string windStr = extractJsonValue(jsonStr, "speed");
    string pressStr = extractJsonValue(jsonStr, "pressure");
    string descStr = extractJsonValue(jsonStr, "description");

    if (!tempStr.empty() && !humStr.empty()) {
        temp = stof(tempStr);

        humidity = stoi(humStr);
        windSpeed = windStr.empty() ? 0.0 : stof(windStr);
        pressure = pressStr.empty() ? 1013 : stoi(pressStr);
        condition = descStr.empty() ? "Clear" : descStr;
        if (!condition.empty()) condition[0] = toupper(condition[0]);

        cout << "[API] ? Real-time data received!" << endl;
        remove("temp_api.json");
        return true;
    }

    cout << "[API] Parsing failed." << endl;
    remove("temp_api.json");
    return false;
}

bool getStatus() const { return isConnected; }
};

// =====
// ? NEW: SYSTEM SETTINGS MODULE

```

```
// =====
class SystemSettings {
private:
    bool autoSave;

    bool metricUnits; // true = Celsius, false = Fahrenheit
    int apiRefreshInterval; // minutes (informational)

    string settingsFile;

    void loadSettings() {
        ifstream file(settingsFile);
        if (file.is_open()) {
            string line;
            while (getline(file, line)) {
                if (line.find("auto_save=") == 0) autoSave = (line.substr(10) ==
"true");

                else if (line.find("metric_units=") == 0) metricUnits =
(line.substr(13) == "true");

                else if (line.find("refresh_interval=") == 0) apiRefreshInterval
= stoi(line.substr(17));
            }
            file.close();
        } else {
            saveSettings(); // Create default
        }
    }

    void saveSettings() {
        ofstream file(settingsFile);
        if (file.is_open()) {
```

```

        file << "auto_save=" << (autoSave ? "true" : "false") << endl;
        file << "metric_units=" << (metricUnits ? "true" : "false") << endl;
        file << "refresh_interval=" << apiRefreshInterval << endl;
        file.close();
    }
}

public:
    SystemSettings() : autoSave(true), metricUnits(true), apiRefreshInterval(10),
settingsFile("system_settings.txt") {
        loadSettings();
    }

    void displaySettings() const {
        cout << "\n===== SYSTEM SETTINGS =====" << endl;
        cout << "1. Auto-Save Data: " << (autoSave ? "ON" : "OFF") << endl;
        cout << "2. Temperature Unit: " << (metricUnits ? "Celsius (°C)" :
"Fahrenheit (°F)") << endl;
        cout << "3. API Refresh Interval: " << apiRefreshInterval << " mins (Free
Tier Limit)" << endl;
        cout << "4. Clear System Cache" << endl;
        cout << "5. View System Info" << endl;
        cout << "6. Back to Admin Menu" << endl;
        cout << "===== " << endl;
    }

    void manage() {
        int choice;

```

```
do {  
    displaySettings();  
    cout << "Enter choice: ";  
    cin >> choice;  
  
    switch(choice) {  
        case 1:  
            autoSave = !autoSave;  
            saveSettings();  
            cout << "Auto-Save toggled to " << (autoSave ? "ON" : "OFF")  
<< endl;  
            break;  
        case 2:  
            metricUnits = !metricUnits;  
            saveSettings();  
            cout << "Units changed to " << (metricUnits ? "Celsius" :  
"Fahrenheit") << endl;  
            break;  
        case 3:  
            cout << "Enter new interval (mins, 5-60): ";  
            int interval;  
            cin >> interval;  
            if (interval >= 5 && interval <= 60) {  
                apiRefreshInterval = interval;  
                saveSettings();  
                cout << "Refresh interval updated!" << endl;  
            } else {  
                cout << "Invalid interval. Must be between 5 and 60." <<  
endl;
```

```
        }  
        break;  
    case 4:  
        cout << "Clearing system cache & temp files..." << endl;  
        remove("temp_api.json");  
        cout << "Cache cleared successfully!" << endl;  
        break;  
    case 5:  
        cout << "\n===== SYSTEM INFO =====" << endl;  
        cout << "Version: 4.1.0" << endl;  
        cout << "Build Date: May 2026" << endl;  
        cout << "API Provider: OpenWeatherMap" << endl;  
        cout << "Language: C++17 (OOP)" << endl;  
        cout << "License: University Project" << endl;  
        cout << "===== " << endl;  
        break;  
    case 6:  
        cout << "Returning to admin menu..." << endl;  
        break;  
    default:  
        cout << "Invalid choice!" << endl;  
    }  
    if (choice != 6) {  
        cout << "\nPress Enter to continue...";  
        cin.ignore(); cin.get();  
    }  
} while (choice != 6);  
}
```

```

    bool isAutoSave() const { return autoSave; }

    bool isMetric() const { return metricUnits; }
};

// =====
// CLASS: WeatherManagementSystem (Main Controller)
// =====

class WeatherManagementSystem {
private:
    vector<Location> locations;
    vector<CurrentWeather> currentWeatherData;
    vector<ForecastWeather> forecastData;
    User* currentUser;
    WeatherAPI api;
    SystemSettings settings; // ? Integrated Settings Module
    const string DATA_FILE = "weather_data.txt";
    const string FORECAST_FILE = "forecast_data.txt";
    const string USERS_FILE = "users.txt";

    string cleanLine(string line) {
        line.erase(remove(line.begin(), line.end(), '\r'), line.end());
        return line;
    }

public:
    WeatherManagementSystem() : currentUser(nullptr) {

```

```
srand(time(0));

loadDataFromFile();

loadForecastFromFile();

if (currentWeatherData.empty()) {
    initializePakistaniData();
}

createUsersFileIfNotExists();
}

~WeatherManagementSystem() {
    if (settings.isAutoSave()) {
        saveDataToFile();
        saveForecastToFile();
    }

    if (currentUser) delete currentUser;
}

void createUsersFileIfNotExists() {
    ifstream check(USERS_FILE);
    if (!check.good()) {
        ofstream create(USERS_FILE);
        create.close();
    }
    check.close();
}
```

```
void initializePakistaniData() {  
    cout << "\n[System] Initializing Pakistani weather data..." << endl;  
    addCity("Karachi", "Pakistan", 30.0, 65, 12.0, 1010, "Sunny");  
    addCity("Lahore", "Pakistan", 33.0, 55, 8.0, 1012, "Clear");  
    addCity("Islamabad", "Pakistan", 28.0, 70, 6.0, 1015, "Rainy");  
    addCity("Peshawar", "Pakistan", 36.0, 40, 10.0, 1008, "Sunny");  
    addCity("Quetta", "Pakistan", 24.0, 30, 18.0, 1020, "Cloudy");  
    addCity("Multan", "Pakistan", 38.0, 35, 11.0, 1005, "Sunny");  
    if (settings.isAutoSave()) saveDataToFile();  
    cout << "[System] Baseline data initialized!" << endl;  
}  
  
void loadDataFromFile() {  
    ifstream file(DATA_FILE);  
    if (!file.is_open()) return;  
  
    string line;  
    int count = 0;  
    while (getline(file, line)) {  
        line = cleanLine(line);  
        if (line.empty()) continue;  
  
        stringstream ss(line);  
        string city, country, condition, date;  
        float temp, wind;  
        int hum, pres;  
        char comma;
```



```

        bool isLive;

        if (getline(ss, city, ',') &&
            getline(ss, country, ',') &&
            ss >> temp >> comma >> hum >> comma >> wind >> comma >> pres >>
comma >> isLive >> comma &&
            getline(ss, condition, ',') &&
            getline(ss, date)) {

            CurrentWeather weather(city, date, temp, hum, wind, pres,
condition);

            weather.setIsLiveData(isLive);
            currentWeatherData.push_back(weather);
            locations.push_back(Location(currentWeatherData.size(), city,
country));

            count++;
        }
    }

    file.close();
}

void saveDataToFile() {
    ofstream file(DATA_FILE);
    if (!file.is_open()) return;

    for (const auto& weather : currentWeatherData) {
        string country = "Unknown";
        for (const auto& loc : locations) {

```

```

        if (loc.getCityName() == weather.getCityName()) {
            country = loc.getCountry();
            break;
        }
    }

    file << weather.getCityName() << ","
        << country << ","
        << weather.getTemperature() << ","
        << weather.getHumidity() << ","
        << weather.getWindSpeed() << ","
        << weather.getPressure() << ","
        << weather.getIsLiveData() << ","
        << weather.getCondition() << ","
        << weather.getDate() << endl;
    }
    file.close();
}

void loadForecastFromFile() {
    ifstream file(FORECAST_FILE);
    if (!file.is_open()) return;
    string line;
    while (getline(file, line)) {
        line = cleanLine(line);
        if (line.empty()) continue;
        stringstream ss(line);
        string city, day, condition, date;

```

```

        float temp, wind;

        int hum, pres;

        char comma;

        if (getline(ss, city, ',') && getline(ss, day, ',') && getline(ss,
date, ',') &&
            ss >> temp >> comma >> hum >> comma >> wind >> comma >> pres >>
comma && getline(ss, condition)) {

            forecastData.push_back(ForecastWeather(city, day, date, temp,
hum, wind, pres, condition));

        }

    }

    file.close();

}

void saveForecastToFile() {
    ofstream file(FORECAST_FILE);

    if (!file.is_open()) return;

    for (const auto& forecast : forecastData) {
        file << forecast.getCityName() << "," << forecast.getForecastDay() <<
", "
        << forecast.getDate() << "," << forecast.getTemperature() << ", "
        << forecast.getHumidity() << "," << forecast.getWindSpeed() <<
", "
        << forecast.getPressure() << "," << forecast.getCondition() <<
endl;

    }

    file.close();

}

void addCity(const string& city, const string& country,

```

```

        float temp, int hum, float wind, int pres, const string&
condition) {
    for (const auto& weather : currentWeatherData) {
        if (weather.getCityName() == city) return;
    }

    time_t now = time(0);
    tm* ltm = localtime(&now);
    string date = to_string(1900 + ltm->tm_year) + "-" +
                  to_string(1 + ltm->tm_mon) + "-" +
                  to_string(ltm->tm_mday);

    CurrentWeather newWeather(city, date, temp, hum, wind, pres, condition);
    currentWeatherData.push_back(newWeather);
    locations.push_back(Location(currentWeatherData.size(), city, country));
}

void viewCurrentWeather(const string& cityName) {
    cout << "\n===== FETCHING REAL-TIME WEATHER =====" << endl;

    float temp; int humidity; float windSpeed; int pressure; string
condition;

    bool apiSuccess = api.fetchData(cityName, temp, humidity, windSpeed,
pressure, condition);

    if (apiSuccess) {
        bool found = false;
        for (auto& weather : currentWeatherData) {

```

```

        if (weather.getCityName() == cityName) {
            weather.setTemperature(temp);
            weather.setHumidity(humidity);
            weather.setWindSpeed(windSpeed);
            weather.setPressure(pressure);
            weather.setCondition(condition);

            weather.setIsLiveData(true);
            weather.checkExtremeWeather();

            time_t now = time(0);
            tm* ltm = localtime(&now);
            string date = to_string(1900 + ltm->tm_year) + "-" +
                        to_string(1 + ltm->tm_mon) + "-" +
                        to_string(ltm->tm_mday);
            weather.setDate(date);

            found = true;
            break;
        }
    }

    if (!found) {
        addCity(cityName, "Pakistan", temp, humidity, windSpeed,
pressure, condition);
    }

    if (settings.isAutoSave()) saveDataToFile();
} else {
    cout << "[System] ? API unavailable. Showing cached data..." << endl;
}

```

```

bool displayed = false;
for (const auto& weather : currentWeatherData) {
    if (weather.getCityName() == cityName) {
        weather.display(settings.isMetric()); // ? Passes unit setting
        displayed = true;
        break;
    }
}

if (!displayed) {
    cout << "City '" << cityName << "' not found." << endl;
}
}

void viewForecast(const string& cityName) {
    cout << "\n===== 5-DAY FORECAST =====" << endl;
    cout << "City: " << cityName << endl;
    bool found = false;
    for (const auto& forecast : forecastData) {
        if (forecast.getCityName() == cityName) {
            forecast.display(settings.isMetric()); // ? Passes unit setting
            found = true;
        }
    }

    if (!found) {
        cout << "No forecast available. Generating sample..." << endl;
    }
}

```

```

        generateSampleForecast(cityName);
    }
}

void generateSampleForecast(const string& cityName) {
    string days[] = {"Monday", "Tuesday", "Wednesday", "Thursday", "Friday"};
    string conditions[] = {"Sunny", "Cloudy", "Rainy", "Stormy", "Clear"};
    srand(time(0));
    for (int i = 0; i < 5; i++) {
        float temp = (rand() % 30) + 10;
        int hum = (rand() % 60) + 30;
        float wind = (rand() % 30);
        int pres = 1000 + (rand() % 30);
        string cond = conditions[rand() % 5];
        time_t now = time(0);
        tm* ltm = localtime(&now);
        string date = to_string(1900 + ltm->tm_year) + "-" +
                     to_string(1 + ltm->tm_mon) + "-" +
                     to_string(ltm->tm_mday + i);
        ForecastWeather forecast(cityName, days[i], date, temp, hum, wind,
pres, cond);
        forecastData.push_back(forecast);
        forecast.display(settings.isMetric());
    }
    if (settings.isAutoSave()) saveForecastToFile();
}

void compareCities(const string& city1, const string& city2) {

```

```

    cout << "\n===== CITY COMPARISON =====" << endl;

    CurrentWeather* w1 = nullptr;
    CurrentWeather* w2 = nullptr;

    for (auto& w : currentWeatherData) {
        if (w.getCityName() == city1) w1 = &w;
        if (w.getCityName() == city2) w2 = &w;
    }

    if (!w1 || !w2) {
        cout << "Error: City not found!" << endl;
        return;
    }

    w1->display(settings.isMetric());
    w2->display(settings.isMetric());

    cout << "\n===== RESULT =====" << endl;

    if (*w1 > *w2) cout << city1 << " is HOTTER by " << fixed <<
setprecision(1) << (w1->getTemperature() - w2->getTemperature()) << " °C" <<
endl;

    else if (*w2 > *w1) cout << city2 << " is HOTTER by " << fixed <<
setprecision(1) << (w2->getTemperature() - w1->getTemperature()) << " °C" <<
endl;

    else cout << "Both have SAME temperature!" << endl;
}

void manageFavourites() {
    if (!currentUser) return;

    Favourite& favs = currentUser->getFavourites();

    int choice;

    do {
        cout << "\n1. View Favourites\n2. Add\n3. Remove\n4. Back\nChoice: ";

```



```

        cin >> choice;

        if (choice == 1) favs.displayFavourites();

        else if (choice == 2) { string c; cout << "City: "; cin >> c;
favs.addFavourite(c); }

        else if (choice == 3) { string c; cout << "City: "; cin >> c;
favs.removeFavourite(c); }

    } while (choice != 4);
}

void registerUser() {
    string u, p;

    cout << "\nRegister - Username: "; cin >> u;
    cout << "Password: "; cin >> p;

    ifstream check(USERS_FILE);

    string line;
    while (getline(check, line)) {

        line = cleanLine(line);

        if (line.find(u + ",") == 0) { cout << "User exists!" << endl;
check.close(); return; }

    }

    check.close();

    User temp(0, u, p);

    ofstream out(USERS_FILE, ios::app);

    out << u << "," << temp.encryptPassword(p) << endl;

    out.close();

    cout << "Registered successfully!" << endl;

}

bool loginUserFromFile(const string& u, const string& p) {

```

```

    ifstream file(USERS_FILE);

    if (!file.is_open()) return false;

    string line;
    while (getline(file, line)) {
        line = cleanLine(line);
        if (line.empty()) continue;

        stringstream ss(line);
        string storedU, storedP;
        getline(ss, storedU, ',');
        getline(ss, storedP, ',');

        if (storedU == u) {
            User temp(0, u, p);
            if (temp.encryptPassword(p) == storedP) {
                currentUser = new User(0, u, p);
                currentUser->setLoginStatus(true);
                cout << "\n? Login Successful! Welcome, " << u << "!" <<
endl;

                file.close();
                return true;
            }
        }
    }

    cout << "Invalid credentials!" << endl;
    file.close();
    return false;
}

void viewAllUsers() {

```

```

ifstream file(USERS_FILE);

cout << "\n===== USERS =====" << endl;

string line; int c = 0;
while (getline(file, line)) {
    line = cleanLine(line);
    if (line.empty()) continue;

    stringstream ss(line);
    string u, p;
    getline(ss, u, ',');
    cout << ++c << ". " << u << endl;
}
file.close();
}

void deleteUser() {
    string target;
    cout << "Delete user: "; cin >> target;
    if (target == "admin") { cout << "Cannot delete admin!" << endl; return;
}

    ifstream in(USERS_FILE);
    vector<string> lines;
    string line; bool found = false;
    while (getline(in, line)) {
        line = cleanLine(line);
        if (line.empty()) continue;
        if (line.find(target + ",") == 0) { found = true; continue; }
        lines.push_back(line);
    }
}

```

```

        in.close();

        if (!found) { cout << "Not found!" << endl; return; }

        ofstream out(USERS_FILE, ios::trunc);
        for (const string& l : lines) out << l << endl;
        out.close();

        cout << "Deleted!" << endl;
    }

void resetUserPassword() {
    string target, newPass;
    cout << "Reset password for: "; cin >> target;
    if (target == "admin") { cout << "Cannot reset admin!" << endl; return; }
    ifstream in(USERS_FILE);
    vector<string> lines;
    string line; bool found = false;

    while (getline(in, line)) {
        line = cleanLine(line);
        if (line.empty()) continue;
        if (line.find(target + ",") == 0) {
            found = true;
            cout << "New password: "; cin >> newPass;
            User temp(0, target, newPass);
            lines.push_back(target + "," + temp.encryptPassword(newPass));
        } else {
            lines.push_back(line);
        }
    }

    in.close();
}

```

```

        if (!found) { cout << "Not found!" << endl; return; }

        ofstream out(USERS_FILE, ios::trunc);
        for (const string& l : lines) out << l << endl;
        out.close();

        cout << "Password reset!" << endl;
    }

    void manageUsers() {
        int choice;
        do {
            cout << "\n1. View Users\n2. Delete User\n3. Reset Password\n4.
Back\nChoice: ";

            cin >> choice;

            if (choice == 1) viewAllUsers();
            else if (choice == 2) deleteUser();
            else if (choice == 3) resetUserPassword();

        } while (choice != 4);
    }

    void loginSystem() {
        int choice;

        cout << "\n1. Login User\n2. Register\n3. Login Admin\n4. Exit\nChoice:
";

        cin >> choice;

        if (choice == 4) exit(0);
        if (choice == 2) { registerUser(); return; }

        string u, p;

        cout << "Username: "; cin >> u;
    }

```

```

    cout << "Password: "; cin >> p;

    if (choice == 3) {
        if (u == "admin" && p == "admin") {
            currentUser = new Admin(1, "admin", "admin", 1);
            currentUser->setLoginStatus(true);
            cout << "\n? ADMIN LOGIN SUCCESSFUL!" << endl;
        } else cout << "Invalid admin credentials!" << endl;
    } else {
        loginUserFromFile(u, p);
    }
}

void run() {
    cout << "\n===== " << endl;
    cout << "    WEATHER MANAGEMENT SYSTEM v4.1" << endl;
    cout << "    REAL-TIME OPENWEATHERMAP API" << endl;
    cout << "    SYSTEM SETTINGS INTEGRATED" << endl;
    cout << "===== " << endl;
    api.connect();

    while (true) {
        if (!currentUser || !currentUser->getLoginStatus()) {
            loginSystem();
            if (!currentUser || !currentUser->getLoginStatus()) continue;
        }

        currentUser->showMenu();

        int choice;

        cout << "\nChoice: ";

```

```

        cin >> choice;

        if (choice == 6) {
            currentUser->logout();
            currentUser->setLoginStatus(false);
            delete currentUser;

            currentUser = nullptr;
            continue;
        }

        if (Admin* admin = dynamic_cast<Admin*>(currentUser)) {
            if (choice == 1) {
                cout << "\nAll Cities:" << endl;
                for(auto& w:currentWeatherData)
w.display(settings.isMetric());
            }

            else if (choice == 2) admin->manageDataSources();
            else if (choice == 3) admin->generateHistoricalReport();
            else if (choice == 4) manageUsers();
            else if (choice == 5) settings.manage(); // ? SYSTEM SETTINGS
MODULE

            else cout << "Invalid!" << endl;
        } else {
            if (choice == 1) { string c; cout << "City: "; cin >> c;
viewCurrentWeather(c); }

            else if (choice == 2) { string c; cout << "City: "; cin >> c;
viewForecast(c); }

            else if (choice == 3) manageFavourites();

```

```
        else if (choice == 4) { string c1, c2; cout << "City 1: "; cin >>
c1; cout << "City 2: "; cin >> c2; compareCities(c1, c2); }

        else if (choice == 5) { cout << "\nHistorical Report: Last 7 days
avg 25°C, 60% humidity." << endl; }

        else cout << "Invalid!" << endl;

    }

    cout << "\nPress Enter..."; cin.ignore(); cin.get();

}

};

int main() {
    WeatherManagementSystem system;

    system.run();

    return 0;
}
```

